

[Lv2] JPA 1주차 학습 노트

JDBC의 고통에서 영속성 컨텍스트의 구원으로

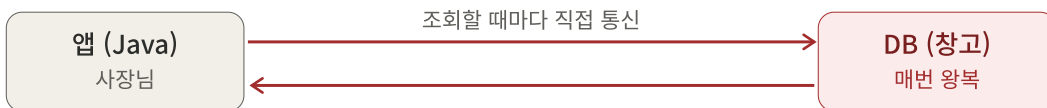
이번 주차의 목표는 "JPA 왜 써요?"에 정의를 외워 답하는 게 아니라, **JDBC의 고통과 그 해결 과정을 직접 겪은 사람으로서 설명할 수 있게 되는 것**이다.

0. 이번 주차가 던지는 질문: "JPA는 대체 왜 쓰는 건데?"

이 강의는 순서가 독특하다. 보통 "JPA는 이렇게 쓴다"부터 가르치는데, 여기서는 **일부러 불편한 옛날 방식(JDBC)을 먼저 겪게 해서 JPA가 왜 태어났는지를 몸으로 느끼게** 한다.

- **JDBC (Java Database Connectivity)**: 자바가 DB와 대화하기 위한 가장 밑바닥의 표준 방법. 전화기로 치면 "번호 누르고, 통화하고, 끊기"를 개발자가 손으로 하나하나 다 하는 방식.

JDBC — 비서 없음



JPA — 영속성 컨텍스트라는 비서 고용



▲ 1주차 전체 배대: JDBC는 앱이 DB를 직접, JPA는 비서(영속성 컨텍스트)가 4가지 기능으로 대행

1. JDBC의 고통 — 왜 이걸 못 견디는가

"유저 한 명 조회하기"라는 단순한 기능을 순수 JDBC로 짜면 **35줄**이 나온다.

```
Connection conn = null;           // ① DB와 연결할 통로
PreparedStatement pstmt = null;    // ② 보낼 SQL을 담는 그릇
ResultSet rs = null;              // ③ DB가 돌려준 결과표
try {
```

```

conn = DriverManager.getConnection(url, id, pw); // 전화 걸기
String sql = "SELECT id, name, email FROM users WHERE id = ?";
pstmt = conn.prepareStatement(sql);
pstmt.setLong(1, id); // ? 자리에 값 채우기
rs = pstmt.executeQuery(); // 실행
if (rs.next()) { // 결과를 한 줄씩 꺼내서
    user = new User();
    user.setName(rs.getString("name")); // 하나하나 손으로 옮겨 담기
}
} catch (SQLException e) { /* ... */ }
finally {
    rs.close(); pstmt.close(); conn.close(); // 반드시 닫아야 함 (순서 중요)
}

```

JDBC 3대 핵심 객체

객체	역할
<code>Connection</code>	자바 앱과 DB를 잇는 "전화선". 연결을 맺는 건 시간이 오래 걸리는 비싼 작업.
<code>PreparedStatement</code>	전화선으로 보낼 "SQL 문장을 담은 그릇". <code>?</code> 자리에 값을 안전하게 채움.
<code>ResultSet</code>	DB가 돌려준 "결과 표". <code>rs.next()</code> 로 한 줄씩 커서를 옮겨가며 값을 꺼냄.

진짜 문제: 숨어 있는 5가지 지뢰 ★ (이번 주차 핵심)

- ① 리소스 누수 (Connection Leak) — DB가 동시에 열어줄 커넥션은 정해져 있다(보통 10~100개). `conn.close()` 를 깜빡하면 회선이 계속 점유된 채 남고, 쌓이면 아무도 DB에 접속 못 하고 서버가 멈춘다. 실무 장애의 단골 원인.
- ② 매번 DB를 다시 조회 (네트워크 낭비) — JDBC는 기억력이 없다. 방금 가져온 데이터를 1초 뒤 다시 달라고 해도 무조건 DB에 다시 물어본다.
- ③ 객체 동일성(Identity) 미보장

```

User user1 = userDao.findById(1L);
User user2 = userDao.findById(1L);
System.out.println(user1 == user2); // false !!

```

DB에서는 같은 한 줄(row)인데, 자바에서는 `new User()` 를 두 번 했기 때문에 서로 다른 객체 두 개가 된다. 데이터가 따로 논다. (`==` 는 "내용이 같냐"가 아니라 "완전히 같은 객체(메모리 주소)냐"를 묻는 비교)

- ④ SQL 의존적 개발 (무한 수정) — 테이블에 컬럼 하나(예: `tel`) 추가하면 INSERT·SELECT·UPDATE 쿼리, 매핑 코드까지 최소 5군데를 손으로 다 고쳐야 한다. 하나 빼먹으면 에러도 없이 조용히 값이 빈다.
- ⑤ 패러다임 불일치 (Impedance Mismatch) — 자바(객체)와 관계형 DB(테이블)는 사고방식이 다르다. 객체는 참조로 연결 (`member.getTeam().getName()`), 테이블은 외래키(FK)로 JOIN해서 합침. 개발자가 매번 "조립"하고 "분해"하는 번역기 노릇을 해야 한다.

용어 정리 (면접 단골)

- DTO (Data Transfer Object): 데이터를 나르는 택배 상자. 필드 + getter/setter만, 기능은 없음.
- DAO (Data Access Object): DB에 직접 접근해 데이터를 넣고 빼는 택배 기사님. 실제 SQL 실행 로직을 가짐.

5가지 고통 요약 — ①리소스 누수 ②네트워크 낭비 ③데이터 불일치 ④반복 노동 ⑤패러다임 불일치. 이 다섯을 한 방에 해결하려고 나온 게 영속성 컨텍스트.

2. 구원자: 영속성 컨텍스트 (Persistence Context)

뜻은 "엔티티를 영구 저장하는 환경" = **앱과 DB 사이에서 데이터를 관리하는 똑똑한 메모리 공간**. 핵심 비유는 **비서**. JDBC 시절엔 사장님(앱)이 창고(DB)를 직접 뛰어다녔다면, JPA에선 비서를 고용해 사장님은 비서에게만 말하고, 비서가 창고를 최소한으로 다녀온다.

용어 계층 정리 (헛갈림 방지)

이름	정체	비유
JPA	"자바에서 DB를 이렇게 다뤄라" 정해진 규칙(표준 명세). 그 자체론 안 돌아감	설계 도면
Hibernate	JPA 규칙을 실제 구현해 돌아가게 만든 구현체	실제 제품
EntityManager	영속성 컨텍스트를 조작하는 도구. <code>em.persist()</code> , <code>em.find()</code>	도구
영속성 컨텍스트	EntityManager가 관리하는 저장소(메모리 공간)	저장소
JpaRepository	쉽게 쓰라고 스프링이 씌운 포장지. <code>save()</code> , <code>findById()</code>	포장지

실무에선 `userRepository.save(user)` 한 줄이지만, 그 안에서 → `EntityManager.persist()` → 영속성 컨텍스트(1차 캐시)에 저장 → 커밋 시점에 SQL 전송, 이 흐름이 돌아간다. 이번 강의는 포장지를 벗기고 알맹이(EntityManager)를 직접 만져보는 중.

3. 필살기 ① 1차 캐시 — "같은 거 왜 두 번 물어?"

1차 캐시의 정체는 거창한 게 아니라 그냥 **자바 Map**이다.

```
Map<식별자, 엔티티> 1차캐시;  
// key = @Id로 지정한 값 (DB의 PK, 예: 1L)  
// value = 실제 User 객체
```

`em.find(User.class, 1L)` 호출 시 JPA는 **DB로 바로 가지 않고 이 Map을 먼저 뒤진다**. 있으면 DB 안 가고 Map에서 바로 반환(쿼리 X), 없으면 DB에서 가져와 Map에 넣고 반환. → 같은 걸 두 번 조회해도 **DB 쿼리는 딱 한 번**. (JDBC 문제 ② 해결)

PK (Primary Key, 기본 키): 테이블에서 각 행을 유일하게 구분하는 값(주민번호 같은 것). `@Id` 가 자바 객체에서 이 PK를 표시하는 표식.

4. 필살기 ② 동일성 보장 — "그놈이 그놈이다"

1차 캐시가 Map이므로 자동으로 따라온다. 같은 PK로 두 번 조회하면 Map의 **바로 그 객체**를 다시 준다.

```
User user1 = em.find(User.class, 1L);  
User user2 = em.find(User.class, 1L);  
System.out.println(user1 == user2); // true !
```

JDBC에서 `false` 였던 게 JPA에선 `true`. 데이터가 따로 놀 일이 없다. (JDBC 문제 ③ 해결) 이걸 **Identity Map 패턴**이라 부른다.

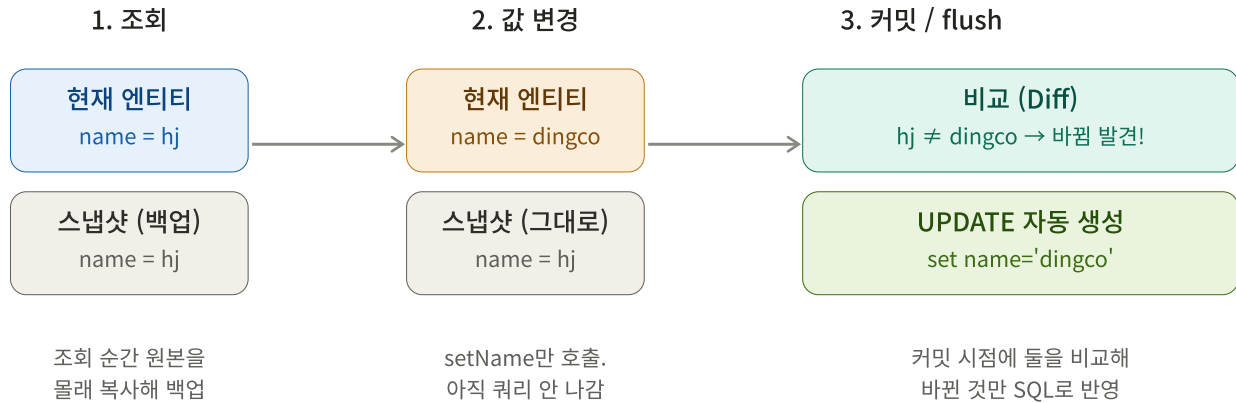
5. 필살기 ③ 변경 감지 (Dirty Checking) — "니가 뭘 바꿨는지 내가 안다"

JPA에는 `update()` 메서드가 없다. 그런데도 데이터가 수정된다.

```
User user = em.find(User.class, 1L); // 조회  
user.setName("새이름"); // 이름만 바꿈. 끝.
```

```
// em.update(user) 같은 거 안 씬!
```

비밀은 **스냅샷(Snapshot)**. 조회하는 순간 그 상태를 몰래 복사해 백업해둔다.



핵심: 개발자는 setName만 부른다. UPDATE는 JPA가 알아서.
자바 List에서 꺼낸 객체를 고치면 List 안에서도 바뀌듯, 다시 저장할 필요가 없다.

▲ 변경 감지: 조회 시 스냅샷 백업 → 커밋 때 현재값과 비교 → 바뀐 것만 자동 UPDATE

비유 — 간단한 교정 편집자: 작가(개발자)에게 원고를 주면서 몰래 복사본(스냅샷)을 챙겨둔다. 작가가 고쳐서 "다 했어!"(commit)하고 가면 복사본과 대조해 바뀐 데만 찾아낸다. **핵심 철학**은 DB 데이터를 자바 컬렉션(List)처럼 다루게 하는 것. List에서 꺼낸 객체를 고치면 끝이듯, JPA도 조회한 엔티티를 고치면 끝(다시 persist 불필요).

⚠ 실무 필수 규칙 2가지

1. **변경 감지는 트랜잭션 안에서만 동작한다.** 트랜잭션이 끝나면 비서가 스냅샷을 버리고 "퇴근"한다. 그 후 값을 바꾸면 반영 안 됨 → **준영속(Detached)** 상태. 수정하려면 반드시 트랜잭션이 살아있는 영역 안에서 바꿔야 한다.
2. 영속(Managed) 상태인 엔티티만 관리 대상이다.

트랜잭션: "한 덩어리로 처리되어야 하는 작업 묶음"(예: 송금 = 출금+입금). **commit** 은 확정, **rollback** 은 전부 취소.

6. 필살기 ④ 쓰기 지연 (Write Behind) — "모아서 한 방에"

택배 1000개를 우체국(왕복 10분)에 보낼 때 — **JDBC 방식**은 1개 포장 → 왕복 → 다시 1개 → 왕복... 1000번(하루가 감). **JPA 방식**은 1000개를 창고(쓰기 지연 저장소)에 쌓아둬 → 마감(commit)에 트럭 한 번에 배송.

DB에서 가장 비싼 비용 중 하나가 **네트워크 왕복(round trip)**. **em.persist()** 를 여러 번 불러도 SQL이 바로 안 나가고 **쓰기 지연 SQL 저장소(ActionQueue)**에 쌓였다가 커밋 시점에 한꺼번에 나간다. (JDBC 문제 ④ 해결)

```
=== 실험 시작 ===
Hibernate: select ... where id=1      <- 조회는 바로 나감
setName 호출 완료 (쿼리 안 나감!)    <- 수정했는데 조용함
=== 커밋 직전 ===                    <- 아직도 조용함
Hibernate: update ... set name=?      <- commit() 순간 광!
```

수정 시점이 아니라 **커밋 시점**에 UPDATE가 나가는 것 = 쓰기 지연 + 변경 감지가 함께 동작하는 증거.

⚠ **주의 (고급)** 쓰기 지연 저장소는 **내 코드 순서대로 SQL을 실행하지 않을 수 있다**. JPA 내부 실행 순서(예: INSERT 먼저, DELETE 나중) 때문에 "삭제 후 같은 값으로 재등록" 같은 코드는 유니크 제약에 걸려 에러가 날 수 있다. 지금 단계에선 "실행 순서가 내 코드 순서와 다를 수 있다"만 기억.

7. 전체를 한 문장으로 & 정리표

JDBC의 5가지 고통(리소스 누수·반복 조회·동일성 불일치·SQL 반복 수정·패러다임 불일치)을 해결하려고 **영속성 컨텍스트**라는 **중간 비서**를 뒀고, 이 비서가 **1차 캐시(기억)·동일성 보장(그놈이 그놈)·변경 감지(알아서 UPDATE)·쓰기 지연(모아서 한 방에)**이라는 4가지 방식으로 DB 접근을 최소화하고 개발을 편하게 만든다.

JDBC의 고통 🙄	영속성 컨텍스트의 해결책 🦸
1. 매번 DB 조회	1차 캐시 — "한 번 본 건 기억해뒀다 바로 줌"
2. 객체 불일치 (<code>==</code> false)	동일성 보장 — "책상에 있는 그놈이 그놈"
3. SQL 무한 수정	변경 감지 — "바뀐 거 알아서 체크하고 고침"
4. 네트워크 낭비	쓰기 지연 — "모았다가 한 번에 처리"
5. 리소스 누수	생명주기 관리 — "문 열고 닫는 건 알아서"

8. 셀프 체크 (복습용)

Q1. JDBC에서 같은 유저를 두 번 조회하면 `user1 == user2` 가 false인데 JPA에선 true인 이유는?

A. 1차 캐시(Map) 때문. 두 번째 조회 시 DB로 안 가고 캐시의 그 객체를 그대로 반환(동일성 보장).

Q2. JPA에 `update()` 가 없는데도 수정되는 이유는?

A. 변경 감지(Dirty Checking). 조회 시 찍어둔 스냅샷과 현재 엔티티를 커밋 시점에 비교해 자동 UPDATE.

Q3. `persist()` 가 바로 안 나가고 커밋 때 몰아 나가는 이유는?

A. 쓰기 지연(Write Behind). ActionQueue에 쌓았다가 커밋 시 한 번에 전송, 네트워크 왕복 절약.